

```

#!/usr/bin/env python3
"""
Enhanced Economic Calendar System - Complete Implementation
Addresses all technical debt optimizations and integration opportunities
"""

import asyncio
import logging
import sqlite3
import aiosqlite
import pandas as pd
import json
import yaml
import smtplib
import aiofiles
import aiohttp
import hashlib
import traceback
from datetime import datetime, timedelta
from pathlib import Path
from typing import Dict, List, Optional, Any, AsyncGenerator, Union, Callable
from dataclasses import dataclass, asdict, field
from enum import Enum
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
from contextlib import asynccontextmanager
import asyncio
import weakref
import gc
from collections import deque
import psutil
import structlog
from pydantic import BaseModel, Field, validator, ValidationError
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler
import concurrent.futures
from twilio.rest import Client as TwilioClient
import numpy as np
from scipy import stats
import plotly.graph_objects as go
import plotly.express as px
from jinja2 import Template
import uuid
import threading

```

```

from queue import Queue
import pickle
import lz4.frame
import asyncio_mqtt
import redis.asyncio as redis

# =====
# 1. DATABASE INDEXING & OPTIMIZATION
# =====

class OptimizedDatabaseManager:
    """Optimized database manager with comprehensive indexing and performance tun

    def __init__(self, db_path: str, max_connections: int = 10):
        self.db_path = Path(db_path)
        self.max_connections = max_connections
        self._connection_pool = asyncio.Queue(maxsize=max_connections)
        self._initialize_pool = False
        self.logger = structlog.get_logger(__name__)

    async def initialize(self):
        """Initialize database with optimized schema and indexes"""
        await self._create_optimized_schema()
        await self._create_performance_indexes()
        await self._initialize_connection_pool()
        await self._configure_performance_settings()

    async def _create_optimized_schema(self):
        """Create optimized database schema with proper data types and constraint
        schema_sql = """
        -- Calendar Events with optimized structure
        CREATE TABLE IF NOT EXISTS calendar_events (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            event_uuid TEXT UNIQUE NOT NULL,
            title TEXT NOT NULL,
            country_code CHAR(3) NOT NULL,
            currency_code CHAR(3) NOT NULL,
            impact_level INTEGER NOT NULL CHECK (impact_level IN (1,2,3)), -- 1=L
            event_date DATE NOT NULL,
            event_time TIME NOT NULL,
            event_timestamp INTEGER NOT NULL, -- Unix timestamp for fast sorting
            trigger_timestamp INTEGER,
            status INTEGER NOT NULL DEFAULT 0, -- 0=PENDING, 1=TRIGGERED, 2=COMPL
            parameter_set_id INTEGER,
            quality_score REAL DEFAULT 0.0,
            forecast_value REAL,
            previous_value REAL,

```

```

    actual_value REAL,
    volatility_impact REAL,
    is_anticipation BOOLEAN DEFAULT FALSE,
    parent_event_uuid TEXT,
    processing_notes TEXT,
    source_file TEXT,
    created_at INTEGER NOT NULL DEFAULT (strftime('%s', 'now')),
    updated_at INTEGER NOT NULL DEFAULT (strftime('%s', 'now')),

    FOREIGN KEY (parent_event_uuid) REFERENCES calendar_events(event_uuid
);

-- Trading Signals with enhanced tracking
CREATE TABLE IF NOT EXISTS trading_signals (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    signal_uuid TEXT UNIQUE NOT NULL,
    event_uuid TEXT NOT NULL,
    symbol TEXT NOT NULL,
    signal_type INTEGER NOT NULL, -- 1=BUY_STOP, 2=SELL_STOP, 3=STRADDLE
    entry_price REAL,
    stop_loss REAL,
    take_profit REAL,
    lot_size REAL NOT NULL,
    status INTEGER NOT NULL DEFAULT 0, -- 0=PENDING, 1=ACTIVE, 2=FILLED,
    strategy_id INTEGER NOT NULL,
    parameter_set_id INTEGER,
    risk_amount REAL,
    account_id TEXT,
    broker_order_id TEXT,
    execution_timestamp INTEGER,
    close_timestamp INTEGER,
    pnl REAL DEFAULT 0.0,
    commission REAL DEFAULT 0.0,
    swap REAL DEFAULT 0.0,
    slippage_pips REAL DEFAULT 0.0,
    execution_time_ms INTEGER,
    metadata TEXT, -- JSON metadata
    created_at INTEGER NOT NULL DEFAULT (strftime('%s', 'now')),

    FOREIGN KEY (event_uuid) REFERENCES calendar_events(event_uuid)
);

-- System Performance Metrics
CREATE TABLE IF NOT EXISTS performance_metrics (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    metric_name TEXT NOT NULL,
    metric_value REAL NOT NULL,

```

```

        metric_unit TEXT,
        component TEXT NOT NULL,
        timestamp INTEGER NOT NULL DEFAULT (strftime('%s', 'now')),
        metadata TEXT
    );

-- Error Logs with categorization
CREATE TABLE IF NOT EXISTS error_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    error_uuid TEXT UNIQUE NOT NULL,
    component TEXT NOT NULL,
    error_level INTEGER NOT NULL, -- 1=DEBUG, 2=INFO, 3=WARNING, 4=ERROR,
    error_code TEXT,
    error_message TEXT NOT NULL,
    stack_trace TEXT,
    context_data TEXT, -- JSON context
    recovery_attempted BOOLEAN DEFAULT FALSE,
    recovery_successful BOOLEAN DEFAULT FALSE,
    resolved BOOLEAN DEFAULT FALSE,
    timestamp INTEGER NOT NULL DEFAULT (strftime('%s', 'now'))
);

-- Configuration Changes Log
CREATE TABLE IF NOT EXISTS config_changes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    change_uuid TEXT UNIQUE NOT NULL,
    config_key TEXT NOT NULL,
    old_value TEXT,
    new_value TEXT,
    changed_by TEXT,
    change_reason TEXT,
    timestamp INTEGER NOT NULL DEFAULT (strftime('%s', 'now'))
);

-- Notification Log
CREATE TABLE IF NOT EXISTS notifications (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    notification_uuid TEXT UNIQUE NOT NULL,
    notification_type INTEGER NOT NULL, -- 1=EMAIL, 2=SMS, 3=WEBHOOK
    recipient TEXT NOT NULL,
    subject TEXT,
    message TEXT NOT NULL,
    priority INTEGER NOT NULL DEFAULT 2, -- 1=LOW, 2=NORMAL, 3=HIGH, 4=CR
    status INTEGER NOT NULL DEFAULT 0, -- 0=PENDING, 1=SENT, 2=FAILED, 3=
    retry_count INTEGER DEFAULT 0,
    max_retries INTEGER DEFAULT 3,
    scheduled_at INTEGER,

```

```

        sent_at INTEGER,
        error_message TEXT,
        metadata TEXT,
        created_at INTEGER NOT NULL DEFAULT (strftime('%s', 'now'))
    );

-- Backtesting Results
CREATE TABLE IF NOT EXISTS backtest_results (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    backtest_uuid TEXT UNIQUE NOT NULL,
    strategy_id INTEGER NOT NULL,
    symbol TEXT NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    total_trades INTEGER NOT NULL,
    winning_trades INTEGER NOT NULL,
    losing_trades INTEGER NOT NULL,
    total_pnl REAL NOT NULL,
    max_drawdown REAL NOT NULL,
    sharpe_ratio REAL,
    profit_factor REAL,
    win_rate REAL,
    avg_win REAL,
    avg_loss REAL,
    max_consecutive_wins INTEGER,
    max_consecutive_losses INTEGER,
    parameters TEXT, -- JSON parameters used
    market_conditions TEXT, -- JSON market context
    created_at INTEGER NOT NULL DEFAULT (strftime('%s', 'now'))
);
"""

async with aiosqlite.connect(self.db_path) as db:
    await db.executescript(schema_sql)
    await db.commit()

async def _create_performance_indexes(self):
    """Create comprehensive indexes for optimal query performance"""
    indexes_sql = """
    -- Calendar Events Indexes
    CREATE INDEX IF NOT EXISTS idx_calendar_events_timestamp ON calendar_event
    CREATE INDEX IF NOT EXISTS idx_calendar_events_status ON calendar_events(
    CREATE INDEX IF NOT EXISTS idx_calendar_events_currency ON calendar_event
    CREATE INDEX IF NOT EXISTS idx_calendar_events_impact ON calendar_events(
    CREATE INDEX IF NOT EXISTS idx_calendar_events_date ON calendar_events(ev
    CREATE INDEX IF NOT EXISTS idx_calendar_events_trigger ON calendar_events
    CREATE INDEX IF NOT EXISTS idx_calendar_events_compound ON calendar_event

```

```
CREATE INDEX IF NOT EXISTS idx_calendar_events_anticipation ON calendar_e
```

```
-- Trading Signals Indexes
```

```
CREATE INDEX IF NOT EXISTS idx_trading_signals_symbol ON trading_signals(
```

```
CREATE INDEX IF NOT EXISTS idx_trading_signals_status ON trading_signals(
```

```
CREATE INDEX IF NOT EXISTS idx_trading_signals_event ON trading_signals(e
```

```
CREATE INDEX IF NOT EXISTS idx_trading_signals_account ON trading_signals
```

```
CREATE INDEX IF NOT EXISTS idx_trading_signals_execution ON trading_signa
```

```
CREATE INDEX IF NOT EXISTS idx_trading_signals_compound ON trading_signal
```

```
-- Performance Metrics Indexes
```

```
CREATE INDEX IF NOT EXISTS idx_performance_metrics_component ON performan
```

```
CREATE INDEX IF NOT EXISTS idx_performance_metrics_timestamp ON performan
```

```
CREATE INDEX IF NOT EXISTS idx_performance_metrics_name ON performance_me
```

```
CREATE INDEX IF NOT EXISTS idx_performance_metrics_compound ON performanc
```

```
-- Error Logs Indexes
```

```
CREATE INDEX IF NOT EXISTS idx_error_logs_component ON error_logs(compone
```

```
CREATE INDEX IF NOT EXISTS idx_error_logs_level ON error_logs(error_level
```

```
CREATE INDEX IF NOT EXISTS idx_error_logs_timestamp ON error_logs(timesta
```

```
CREATE INDEX IF NOT EXISTS idx_error_logs_resolved ON error_logs(resolved
```

```
-- Notifications Indexes
```

```
CREATE INDEX IF NOT EXISTS idx_notifications_type ON notifications(notifi
```

```
CREATE INDEX IF NOT EXISTS idx_notifications_status ON notifications(stat
```

```
CREATE INDEX IF NOT EXISTS idx_notifications_scheduled ON notifications(s
```

```
CREATE INDEX IF NOT EXISTS idx_notifications_priority ON notifications(pr
```

```
-- Backtesting Indexes
```

```
CREATE INDEX IF NOT EXISTS idx_backtest_strategy ON backtest_results(stra
```

```
CREATE INDEX IF NOT EXISTS idx_backtest_symbol ON backtest_results(symbol
```

```
CREATE INDEX IF NOT EXISTS idx_backtest_date_range ON backtest_results(st
```

```
""""
```

```
async with aiosqlite.connect(self.db_path) as db:
```

```
    await db.executescript(indexes_sql)
```

```
    await db.commit()
```

```
async def _configure_performance_settings(self):
```

```
    """Configure SQLite for optimal performance"""
```

```
    performance_sql = """
```

```
PRAGMA journal_mode = WAL;
```

```
PRAGMA synchronous = NORMAL;
```

```
PRAGMA cache_size = 10000;
```

```
PRAGMA temp_store = MEMORY;
```

```
PRAGMA mmap_size = 268435456; -- 256MB
```

```
PRAGMA optimize;
```

```

        """

        async with aiomysql.connect(self.db_path) as db:
            await db.executescript(performance_sql)
            await db.commit()

    async def _initialize_connection_pool(self):
        """Initialize connection pool for better concurrency"""
        for _ in range(self.max_connections):
            conn = await aiomysql.connect(self.db_path)
            conn.row_factory = aiomysql.Row
            await self._connection_pool.put(conn)
        self._initialize_pool = True

    @asynccontextmanager
    async def get_connection(self):
        """Get connection from pool"""
        if not self._initialize_pool:
            await self._initialize_connection_pool()

        conn = await self._connection_pool.get()
        try:
            yield conn
        finally:
            await self._connection_pool.put(conn)

# =====
# 2. FULL ASYNC PROCESSING IMPLEMENTATION
# =====

class AsyncEventProcessor:
    """Fully async event processing with concurrent pipeline stages"""

    def __init__(self, db_manager: OptimizedDatabaseManager, max_workers: int = 1):
        self.db_manager = db_manager
        self.max_workers = max_workers
        self.logger = structlog.get_logger(__name__)
        self._processing_queue = asyncio.Queue(maxsize=1000)
        self._signal_queue = asyncio.Queue(maxsize=1000)
        self._workers = []
        self._running = False

    async def start(self):
        """Start async processing workers"""
        self._running = True

        # Start processing workers

```

```

        for i in range(self.max_workers):
            worker = asyncio.create_task(self._process_events_worker(f"worker-{i}")
            self._workers.append(worker)

        # Start signal generation worker
        signal_worker = asyncio.create_task(self._generate_signals_worker())
        self._workers.append(signal_worker)

        self.logger.info(f"Started {len(self._workers)} async workers")

    async def stop(self):
        """Stop all workers gracefully"""
        self._running = False

        # Cancel all workers
        for worker in self._workers:
            worker.cancel()

        # Wait for workers to finish
        await asyncio.gather(*self._workers, return_exceptions=True)
        self.logger.info("All async workers stopped")

    async def process_calendar_file(self, file_path: Path) -> Dict[str, Any]:
        """Process calendar file with full async pipeline"""
        start_time = asyncio.get_event_loop().time()

        try:
            # Stream process large files
            events = []
            async for event_batch in self._stream_parse_csv(file_path):
                # Add to processing queue
                for event in event_batch:
                    await self._processing_queue.put(event)

            # Wait for processing to complete
            await self._processing_queue.join()

            processing_time = asyncio.get_event_loop().time() - start_time

            return {
                "status": "success",
                "events_processed": len(events),
                "processing_time": processing_time,
                "file_path": str(file_path)
            }

        except Exception as e:

```

```

        self.logger.error(f"Error processing calendar file: {e}", file_path=s
        return {
            "status": "error",
            "error": str(e),
            "file_path": str(file_path)
        }

    async def _stream_parse_csv(self, file_path: Path, batch_size: int = 100) ->
        """Stream parse large CSV files in batches"""
        async with aiofiles.open(file_path, 'r', encoding='utf-8') as f:
            header_line = await f.readline()
            headers = [h.strip() for h in header_line.split(',')]

            batch = []
            async for line in f:
                if not line.strip():
                    continue

                values = [v.strip() for v in line.split(',')]
                if len(values) == len(headers):
                    event_data = dict(zip(headers, values))
                    batch.append(event_data)

                    if len(batch) >= batch_size:
                        yield batch
                        batch = []

            if batch:
                yield batch

    async def _process_events_worker(self, worker_id: str):
        """Async worker for processing events"""
        self.logger.info(f"Started event processing worker: {worker_id}")

        while self._running:
            try:
                # Get event from queue with timeout
                event_data = await asyncio.wait_for(
                    self._processing_queue.get(),
                    timeout=1.0
                )

                # Process event
                processed_event = await self._process_single_event(event_data)

                if processed_event:
                    # Add to signal generation queue

```

```

        await self._signal_queue.put(processed_event)

        # Mark task as done
        self._processing_queue.task_done()

    except asyncio.TimeoutError:
        continue
    except Exception as e:
        self.logger.error(f"Error in worker {worker_id}: {e}")
        self._processing_queue.task_done()

async def _process_single_event(self, event_data: Dict[str, Any]) -> Optional
    """Process single event with validation and enhancement"""
    try:
        # Validate event data
        if not self._validate_event_data(event_data):
            return None

        # Transform and enhance
        event = await self._transform_event(event_data)

        # Generate anticipation events
        anticipation_events = await self._generate_anticipation_events(event)

        # Save to database
        async with self.db_manager.get_connection() as conn:
            # Save main event
            await self._save_event(conn, event)

            # Save anticipation events
            for ant_event in anticipation_events:
                await self._save_event(conn, ant_event)

        return event

    except Exception as e:
        self.logger.error(f"Error processing event: {e}", event_data=event_data)
        return None

def _validate_event_data(self, event_data: Dict[str, Any]) -> bool:
    """Validate event data completeness and format"""
    required_fields = ['title', 'country', 'date', 'time', 'impact']
    return all(field in event_data and event_data[field] for field in require

async def _transform_event(self, event_data: Dict[str, Any]) -> Dict[str, Any]
    """Transform raw event data to standardized format"""
    # Implementation details...

```

```

        return event_data

    async def _generate_anticipation_events(self, event: Dict[str, Any]) -> List[
        """Generate anticipation events for main event"""
        # Implementation details...
        return []

# =====
# 3. ENHANCED ERROR RECOVERY MECHANISMS
# =====

class EnhancedErrorRecovery:
    """Advanced error recovery with automatic healing and circuit breakers"""

    def __init__(self, db_manager: OptimizedDatabaseManager):
        self.db_manager = db_manager
        self.logger = structlog.get_logger(__name__)
        self._circuit_breakers = {}
        self._recovery_strategies = {}
        self._error_patterns = {}
        self._setup_recovery_strategies()

    def _setup_recovery_strategies(self):
        """Setup recovery strategies for different error types"""
        self._recovery_strategies = {
            'database_connection': self._recover_database_connection,
            'file_processing': self._recover_file_processing,
            'network_timeout': self._recover_network_timeout,
            'memory_overflow': self._recover_memory_overflow,
            'configuration_error': self._recover_configuration_error,
            'signal_generation': self._recover_signal_generation
        }

    async def handle_error(self, error: Exception, context: Dict[str, Any]) -> bool:
        """Handle error with automatic recovery attempt"""
        error_uuid = str(uuid.uuid4())
        error_type = self._classify_error(error)

        try:
            # Log error
            await self._log_error(error_uuid, error, context, error_type)

            # Check circuit breaker
            if await self._check_circuit_breaker(error_type):
                self.logger.warning(f"Circuit breaker open for {error_type}")
                return False

```

```

        # Attempt recovery
        recovery_successful = await self._attempt_recovery(error_type, error,

        # Update circuit breaker
        await self._update_circuit_breaker(error_type, recovery_successful)

        # Log recovery result
        await self._log_recovery_result(error_uuid, recovery_successful)

        return recovery_successful

    except Exception as recovery_error:
        self.logger.error(f"Error in error recovery: {recovery_error}")
        return False

def _classify_error(self, error: Exception) -> str:
    """Classify error type for appropriate recovery strategy"""
    error_str = str(error).lower()

    if 'database' in error_str or 'sqlite' in error_str:
        return 'database_connection'
    elif 'file' in error_str or 'permission' in error_str:
        return 'file_processing'
    elif 'timeout' in error_str or 'connection' in error_str:
        return 'network_timeout'
    elif 'memory' in error_str or 'overflow' in error_str:
        return 'memory_overflow'
    elif 'config' in error_str or 'validation' in error_str:
        return 'configuration_error'
    elif 'signal' in error_str:
        return 'signal_generation'
    else:
        return 'unknown'

async def _check_circuit_breaker(self, error_type: str) -> bool:
    """Check if circuit breaker is open for error type"""
    breaker = self._circuit_breakers.get(error_type, {
        'failure_count': 0,
        'last_failure': None,
        'state': 'closed' # closed, open, half_open
    })

    if breaker['state'] == 'open':
        # Check if enough time has passed to try again
        if breaker['last_failure']:
            time_since_failure = datetime.now().timestamp() - breaker['last_f
            if time_since_failure > 300: # 5 minutes

```

```

        breaker['state'] = 'half_open'
        self._circuit_breakers[error_type] = breaker
        return False
    return True

return False

async def _update_circuit_breaker(self, error_type: str, recovery_successful:
    """Update circuit breaker state based on recovery result"""
    breaker = self._circuit_breakers.get(error_type, {
        'failure_count': 0,
        'last_failure': None,
        'state': 'closed'
    })

    if recovery_successful:
        if breaker['state'] == 'half_open':
            breaker['state'] = 'closed'
            breaker['failure_count'] = 0
        else:
            breaker['failure_count'] += 1
            breaker['last_failure'] = datetime.now().timestamp()

            if breaker['failure_count'] >= 5:
                breaker['state'] = 'open'

    self._circuit_breakers[error_type] = breaker

async def _attempt_recovery(self, error_type: str, error: Exception, context:
    """Attempt recovery using appropriate strategy"""
    recovery_strategy = self._recovery_strategies.get(error_type)

    if recovery_strategy:
        try:
            return await recovery_strategy(error, context)
        except Exception as e:
            self.logger.error(f"Recovery strategy failed: {e}")
            return False
    else:
        self.logger.warning(f"No recovery strategy for error type: {error_type}")
        return False

async def _recover_database_connection(self, error: Exception, context: Dict[
    """Recover from database connection issues"""
    try:
        # Reinitialize database connection
        await self.db_manager.initialize()

```

```

        # Test connection
        async with self.db_manager.get_connection() as conn:
            await conn.execute("SELECT 1")

        self.logger.info("Database connection recovered successfully")
        return True

    except Exception as e:
        self.logger.error(f"Database recovery failed: {e}")
        return False

    async def _recover_file_processing(self, error: Exception, context: Dict[str,
        """Recover from file processing issues"""
        file_path = context.get('file_path')
        if not file_path:
            return False

        try:
            # Check file permissions
            if not Path(file_path).exists():
                self.logger.error(f"File does not exist: {file_path}")
                return False

            # Try to read file with different encoding
            encodings = ['utf-8', 'latin-1', 'cp1252']
            for encoding in encodings:
                try:
                    async with aiofiles.open(file_path, 'r', encoding=encoding) as f:
                        await f.read(100) # Test read
                    self.logger.info(f"File recovery successful with encoding: {encoding}")
                    return True
                except UnicodeDecodeError:
                    continue

            return False

        except Exception as e:
            self.logger.error(f"File processing recovery failed: {e}")
            return False

# =====
# 4. CONFIGURATION VALIDATION & HOT-RELOADING
# =====

class ConfigManager:
    """Advanced configuration manager with validation and hot-reloading"""

```

```

def __init__(self, config_path: str):
    self.config_path = Path(config_path)
    self.logger = structlog.get_logger(__name__)
    self._config = {}
    self._schema = {}
    self._observers = []
    self._change_callbacks = []
    self._file_observer = None
    self._load_schema()

def _load_schema(self):
    """Load configuration schema for validation"""
    self._schema = {
        "type": "object",
        "properties": {
            "calendar_auto_import": {"type": "boolean"},
            "import_day": {"type": "integer", "minimum": 0, "maximum": 6},
            "import_hour": {"type": "integer", "minimum": 0, "maximum": 23},
            "anticipation_hours": {
                "type": "array",
                "items": {"type": "integer", "minimum": 1, "maximum": 24}
            },
            "trigger_offsets": {
                "type": "object",
                "properties": {
                    "EMO-E": {"type": "integer"},
                    "EMO-A": {"type": "integer"},
                    "EQT-OPEN": {"type": "integer"},
                    "ANTICIPATION": {"type": "integer"}
                }
            },
            "parameter_sets": {
                "type": "object",
                "patternProperties": {
                    "^[1-4]$": {
                        "type": "object",
                        "properties": {
                            "lot_size": {"type": "number", "minimum": 0.01},
                            "buy_distance": {"type": "number", "minimum": 1},
                            "sell_distance": {"type": "number", "minimum": 1},
                            "stop_loss": {"type": "number", "minimum": 1},
                            "take_profit": {"type": "number", "minimum": 1}
                        },
                        "required": ["lot_size", "buy_distance", "sell_distan
                    }
                }
            }
        }
    }

```

```

    },
    "notification_settings": {
        "type": "object",
        "properties": {
            "email_enabled": {"type": "boolean"},
            "sms_enabled": {"type": "boolean"},
            "webhook_enabled": {"type": "boolean"},
            "smtp_settings": {
                "type": "object",
                "properties": {
                    "host": {"type": "string"},
                    "port": {"type": "integer"},
                    "username": {"type": "string"},
                    "password": {"type": "string"},
                    "use_tls": {"type": "boolean"}
                }
            }
        }
    }
},
    "required": ["calendar_auto_import", "import_day", "import_hour"]
}

```

```

async def load_config(self) -> Dict[str, Any]:
    """Load and validate configuration"""
    try:
        if self.config_path.exists():
            async with aiofiles.open(self.config_path, 'r') as f:
                content = await f.read()
                config = yaml.safe_load(content)
        else:
            config = self._get_default_config()
            await self.save_config(config)

        # Validate configuration
        validation_result = await self._validate_config(config)
        if not validation_result['valid']:
            raise ValueError(f"Invalid configuration: {validation_result['err']}")

        self._config = config
        return config

    except Exception as e:
        self.logger.error(f"Error loading configuration: {e}")
        return self._get_default_config()

async def save_config(self, config: Dict[str, Any]):

```

```

"""Save configuration to file"""
try:
    # Validate before saving
    validation_result = await self._validate_config(config)
    if not validation_result['valid']:
        raise ValueError(f"Cannot save invalid configuration: {validation_result}")

    # Create backup of current config
    if self.config_path.exists():
        backup_path = self.config_path.with_suffix(f'.bak.{int(datetime.now().timestamp())}')
        async with aiofiles.open(self.config_path, 'r') as src:
            async with aiofiles.open(backup_path, 'w') as dst:
                content = await src.read()
                await dst.write(content)

    # Save new configuration
    async with aiofiles.open(self.config_path, 'w') as f:
        yaml_content = yaml.dump(config, default_flow_style=False, sort_keys=False)
        await f.write(yaml_content)

    self._config = config

    # Notify observers of change
    await self._notify_config_change(config)

except Exception as e:
    self.logger.error(f"Error saving configuration: {e}")
    raise

async def _validate_config(self, config: Dict[str, Any]) -> Dict[str, Any]:
    """Validate configuration against schema"""
    try:
        from jsonschema import validate, ValidationError as JsonValidationError

        validate(instance=config, schema=self._schema)

        # Additional business logic validation
        errors = []

        # Validate anticipation hours
        if 'anticipation_hours' in config:
            ant_hours = config['anticipation_hours']
            if len(ant_hours) > 5:
                errors.append("Too many anticipation hours (max 5)")
            if not all(isinstance(h, int) and 1 <= h <= 24 for h in ant_hours):
                errors.append("Invalid anticipation hours")
    
```

```

        # Validate parameter sets
        if 'parameter_sets' in config:
            param_sets = config['parameter_sets']
            if len(param_sets) != 4:
                errors.append("Must have exactly 4 parameter sets")

        return {"valid": len(errors) == 0, "errors": errors}

    except JsonValidationError as e:
        return {"valid": False, "errors": [str(e)]}
    except Exception as e:
        return {"valid": False, "errors": [f"Validation error: {str(e)}"]}

def start_hot_reload(self):
    """Start file watching for hot configuration reloading"""
    class ConfigChangeHandler(FileSystemEventHandler):
        def __init__(self, config_manager):
            self.config_manager = config_manager

        def on_modified(self, event):
            if event.src_path == str(self.config_manager.config_path):
                asyncio.create_task(self.config_manager._reload_config())

    if self._file_observer is None:
        handler = ConfigChangeHandler(self)
        self._file_observer = Observer()
        self._file_observer.schedule(handler, str(self.config_path.parent), r
        self._file_observer.start()
        self.logger.info("Started configuration hot-reload monitoring")

    async def _reload_config(self):
        """Reload configuration from file"""
        try:
            old_config = self._config.copy()
            new_config = await self.load_config()

            # Check for changes
            if old_config != new_config:
                self.logger.info("Configuration changed, reloading...")
                await self._notify_config_change(new_config)

        except Exception as e:
            self.logger.error(f"Error reloading configuration: {e}")

# =====
# 5. MEMORY OPTIMIZATION & STREAMING
# =====

```

```

class MemoryOptimizedProcessor:
    """Memory-optimized processor for large calendar files"""

    def __init__(self, chunk_size: int = 1000, memory_limit_mb: int = 100):
        self.chunk_size = chunk_size
        self.memory_limit_bytes = memory_limit_mb * 1024 * 1024
        self.logger = structlog.get_logger(__name__)
        self._memory_monitor = None

    async def process_large_file(self, file_path: Path) -> AsyncGenerator[Dict[str, str], None]:
        """Process large files with memory optimization"""
        self._start_memory_monitoring()

        try:
            # Use streaming CSV reader
            async for chunk in self._stream_csv_chunks(file_path):
                # Process chunk in memory-efficient way
                processed_chunk = await self._process_chunk_optimized(chunk)

                # Check memory usage
                if self._check_memory_limit():
                    await self._free_memory()

                yield processed_chunk

        finally:
            self._stop_memory_monitoring()

    async def _stream_csv_chunks(self, file_path: Path) -> AsyncGenerator[List[Dict[str, str]], None]:
        """Stream CSV file in memory-efficient chunks"""
        async with aiofiles.open(file_path, 'r', encoding='utf-8') as f:
            # Read header
            header_line = await f.readline()
            headers = [h.strip().strip('"') for h in header_line.split(',')]

            chunk = []
            line_count = 0

            async for line in f:
                line = line.strip()
                if not line:
                    continue

                try:
                    # Parse CSV line efficiently
                    values = self._parse_csv_line(line)
                
```

```

        if len(values) == len(headers):
            row_data = dict(zip(headers, values))
            chunk.append(row_data)
            line_count += 1

        if len(chunk) >= self.chunk_size:
            yield chunk
            chunk = []

        # Force garbage collection every 10 chunks
        if line_count % (self.chunk_size * 10) == 0:
            gc.collect()

    except Exception as e:
        self.logger.warning(f"Error parsing line {line_count}: {e}")
        continue

    if chunk:
        yield chunk

def _parse_csv_line(self, line: str) -> List[str]:
    """Efficiently parse CSV line handling quotes and commas"""
    values = []
    current_value = ""
    in_quotes = False

    i = 0
    while i < len(line):
        char = line[i]

        if char == '"':
            if in_quotes and i + 1 < len(line) and line[i + 1] == '"':
                # Escaped quote
                current_value += '"'
                i += 1
            else:
                in_quotes = not in_quotes
        elif char == ',' and not in_quotes:
            values.append(current_value.strip())
            current_value = ""
        else:
            current_value += char

        i += 1

    values.append(current_value.strip())
    return values

```

```

async def _process_chunk_optimized(self, chunk: List[Dict]) -> List[Dict]:
    """Process chunk with memory optimization"""
    processed = []

    for row in chunk:
        try:
            # Process row efficiently
            processed_row = self._process_row_minimal_copy(row)
            if processed_row:
                processed.append(processed_row)

        except Exception as e:
            self.logger.warning(f"Error processing row: {e}")
            continue

    return processed

def _process_row_minimal_copy(self, row: Dict[str, Any]) -> Optional[Dict[str, Any]]:
    """Process row with minimal memory copying"""
    # Extract only necessary fields to minimize memory usage
    essential_fields = ['title', 'country', 'date', 'time', 'impact', 'forecast']

    processed = {}
    for field in essential_fields:
        if field in row:
            processed[field] = row[field]

    # Validate essential data
    if not all(processed.get(f) for f in ['title', 'country', 'date', 'time']):
        return None

    return processed

def _start_memory_monitoring(self):
    """Start monitoring memory usage"""
    self._memory_monitor = psutil.Process()

def _check_memory_limit(self) -> bool:
    """Check if memory limit is exceeded"""
    if self._memory_monitor:
        memory_info = self._memory_monitor.memory_info()
        return memory_info.rss > self.memory_limit_bytes
    return False

async def _free_memory(self):
    """Force memory cleanup"""

```

```

        gc.collect()
        await asyncio.sleep(0.1)  # Allow garbage collection to complete

# =====
# 6. NOTIFICATION SYSTEM
# =====

class NotificationSystem:
    """Comprehensive notification system with multiple channels"""

    def __init__(self, db_manager: OptimizedDatabaseManager, config: Dict[str, An
        self.db_manager = db_manager
        self.config = config
        self.logger = structlog.get_logger(__name__)
        self._notification_queue = asyncio.Queue()
        self._workers = []
        self._running = False

        # Initialize notification channels
        self._email_client = None
        self._sms_client = None
        self._webhook_session = None

    async def start(self):
        """Start notification system"""
        await self._initialize_clients()
        self._running = True

        # Start notification workers
        for i in range(3):  # 3 workers for parallel processing
            worker = asyncio.create_task(self._notification_worker(f"notification
            self._workers.append(worker)

        self.logger.info("Notification system started")

    async def stop(self):
        """Stop notification system"""
        self._running = False

        for worker in self._workers:
            worker.cancel()

        await asyncio.gather(*self._workers, return_exceptions=True)

        if self._webhook_session:
            await self._webhook_session.close()

```

```

async def send_notification(self, notification_type: str, recipient: str,
                           subject: str, message: str, priority: int = 2,
                           metadata: Dict[str, Any] = None):
    """Send notification through appropriate channel"""
    notification = {
        'uuid': str(uuid.uuid4()),
        'type': notification_type,
        'recipient': recipient,
        'subject': subject,
        'message': message,
        'priority': priority,
        'metadata': json.dumps(metadata or {}),
        'scheduled_at': int(datetime.now().timestamp())
    }

    # Save to database
    async with self.db_manager.get_connection() as conn:
        await conn.execute("""
            INSERT INTO notifications
            (notification_uuid, notification_type, recipient, subject, message,
             priority, scheduled_at, metadata)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)
        """, (
            notification['uuid'],
            1 if notification_type == 'email' else 2 if notification_type ==
            notification['recipient'], notification['subject'], notification[
            notification['priority'], notification['scheduled_at'], notificat
        ))

    # Add to processing queue
    await self._notification_queue.put(notification)

async def send_critical_alert(self, title: str, message: str, details: Dict[s
    """Send critical system alert to all configured channels"""
    recipients = self.config.get('notification_settings', {}).get('alert_reci

    for recipient in recipients:
        if '@' in recipient: # Email
            await self.send_notification('email', recipient, f"CRITICAL ALERT
                                     message, priority=4, metadata=details)
        elif recipient.startswith('+'): # SMS
            await self.send_notification('sms', recipient, title,
                                     message[:160], priority=4, metadata=de

async def _notification_worker(self, worker_id: str):
    """Notification processing worker"""
    while self._running:

```

```

try:
    notification = await asyncio.wait_for(
        self._notification_queue.get(), timeout=1.0
    )

    success = await self._send_notification(notification)
    await self._update_notification_status(notification['uuid'], succ

except asyncio.TimeoutError:
    continue
except Exception as e:
    self.logger.error(f"Error in notification worker {worker_id}: {e}

async def _send_notification(self, notification: Dict[str, Any]) -> bool:
    """Send individual notification"""
    try:
        notification_type = notification['type']

        if notification_type == 'email':
            return await self._send_email(notification)
        elif notification_type == 'sms':
            return await self._send_sms(notification)
        elif notification_type == 'webhook':
            return await self._send_webhook(notification)

        return False

    except Exception as e:
        self.logger.error(f"Error sending notification: {e}")
        return False

async def _send_email(self, notification: Dict[str, Any]) -> bool:
    """Send email notification"""
    try:
        email_config = self.config.get('notification_settings', {}).get('smtp

        if not email_config:
            self.logger.warning("Email configuration not found")
            return False

        # Create message
        msg = MIMEMultipart()
        msg['From'] = email_config['username']
        msg['To'] = notification['recipient']
        msg['Subject'] = notification['subject']

        # Add body

```

```

        body = MimeType(notification['message'], 'plain')
        msg.attach(body)

        # Send email
        with smtplib.SMTP(email_config['host'], email_config['port']) as serv
            if email_config.get('use_tls', True):
                server.starttls()
            server.login(email_config['username'], email_config['password'])
            server.send_message(msg)

        return True

    except Exception as e:
        self.logger.error(f"Error sending email: {e}")
        return False

    async def _send_sms(self, notification: Dict[str, Any]) -> bool:
        """Send SMS notification"""
        try:
            sms_config = self.config.get('notification_settings', {}).get('twilio

            if not sms_config:
                self.logger.warning("SMS configuration not found")
                return False

            if not self._sms_client:
                self._sms_client = TwilioClient(
                    sms_config['account_sid'],
                    sms_config['auth_token']
                )

            message = self._sms_client.messages.create(
                body=notification['message'],
                from_=sms_config['from_number'],
                to=notification['recipient']
            )

            return True

        except Exception as e:
            self.logger.error(f"Error sending SMS: {e}")
            return False

# =====
# 7. RISK MANAGEMENT INTEGRATION
# =====

```

```

class RiskManager:
    """Advanced risk management with position sizing and portfolio limits"""

    def __init__(self, db_manager: OptimizedDatabaseManager):
        self.db_manager = db_manager
        self.logger = structlog.get_logger(__name__)
        self._position_limits = {}
        self._portfolio_limits = {}

    async def calculate_position_size(self, signal: Dict[str, Any], account_info:
        """Calculate optimal position size based on risk parameters"""
        try:
            account_balance = account_info.get('balance', 10000)
            risk_per_trade = account_info.get('risk_per_trade_pct', 2.0) / 100
            max_risk_amount = account_balance * risk_per_trade

            # Calculate stop loss distance in account currency
            symbol = signal['symbol']
            stop_loss_pips = signal.get('stop_loss', 40)
            pip_value = await self._get_pip_value(symbol, account_info.get('curre

            # Calculate position size
            risk_amount_per_pip = max_risk_amount / stop_loss_pips
            lot_size = risk_amount_per_pip / pip_value

            # Apply position limits
            max_lot_size = self._position_limits.get(symbol, {}).get('max_lot_siz
            lot_size = min(lot_size, max_lot_size)

            # Round to broker minimum
            min_lot_size = 0.01
            lot_size = max(min_lot_size, round(lot_size / min_lot_size) * min_lot

            return {
                'recommended_lot_size': lot_size,
                'risk_amount': lot_size * pip_value * stop_loss_pips,
                'risk_percentage': (lot_size * pip_value * stop_loss_pips / accou
                'max_risk_amount': max_risk_amount,
                'pip_value': pip_value
            }

        except Exception as e:
            self.logger.error(f"Error calculating position size: {e}")
            return {'recommended_lot_size': 0.01, 'risk_amount': 0, 'risk_percent

    async def validate_trade_risk(self, signal: Dict[str, Any], account_id: str)
        """Validate trade against risk management rules"""

```

```

try:
    # Get current portfolio exposure
    portfolio_exposure = await self._get_portfolio_exposure(account_id)

    # Calculate new trade risk
    trade_risk = signal.get('risk_amount', 0)
    new_total_risk = portfolio_exposure.get('total_risk', 0) + trade_risk

    # Check portfolio limits
    max_portfolio_risk = self._portfolio_limits.get(account_id, {}).get('
    max_daily_trades = self._portfolio_limits.get(account_id, {}).get('ma

    # Get today's trade count
    today_trades = await self._get_today_trade_count(account_id)

    # Validation checks
    checks = {
        'portfolio_risk_ok': new_total_risk <= max_portfolio_risk,
        'daily_trades_ok': today_trades < max_daily_trades,
        'correlation_ok': await self._check_correlation_limits(signal, ac
        'drawdown_ok': await self._check_drawdown_limits(account_id)
    }

    return {
        'approved': all(checks.values()),
        'checks': checks,
        'current_risk': portfolio_exposure.get('total_risk', 0),
        'new_total_risk': new_total_risk,
        'max_risk': max_portfolio_risk,
        'today_trades': today_trades,
        'max_daily_trades': max_daily_trades
    }

except Exception as e:
    self.logger.error(f"Error validating trade risk: {e}")
    return {'approved': False, 'error': str(e)}

# =====
# 8. PORTFOLIO MANAGEMENT
# =====

class PortfolioManager:
    """Multi-account portfolio management and signal distribution"""

    def __init__(self, db_manager: OptimizedDatabaseManager):
        self.db_manager = db_manager
        self.logger = structlog.get_logger(__name__)

```

```

self._account_configs = {}

async def distribute_signal(self, master_signal: Dict[str, Any]) -> List[Dict
    """Distribute trading signal to multiple accounts"""
    distributed_signals = []

    try:
        # Get active accounts
        accounts = await self._get_active_accounts()

        for account in accounts:
            # Check if account should receive this signal
            if await self._should_send_signal(account, master_signal):
                # Customize signal for account
                account_signal = await self._customize_signal_for_account(
                    master_signal, account
                )

                if account_signal:
                    distributed_signals.append(account_signal)

        # Log distribution
        self.logger.info(f"Distributed signal to {len(distributed_signals)} a

    return distributed_signals

except Exception as e:
    self.logger.error(f"Error distributing signal: {e}")
    return []

async def _get_active_accounts(self) -> List[Dict[str, Any]]:
    """Get list of active trading accounts"""
    # This would typically come from a database or configuration
    return [
        {
            'account_id': 'account_1',
            'account_type': 'live',
            'currency': 'USD',
            'balance': 10000,
            'risk_profile': 'conservative',
            'active': True
        },
        {
            'account_id': 'account_2',
            'account_type': 'demo',
            'currency': 'EUR',
            'balance': 5000,

```

```

        'risk_profile': 'aggressive',
        'active': True
    }
]

async def _should_send_signal(self, account: Dict[str, Any], signal: Dict[str, Any]) -> Optional[Dict[str, Any]]:
    """Determine if signal should be sent to specific account"""
    # Account-specific filtering logic
    if not account.get('active', False):
        return False

    # Risk profile filtering
    risk_profile = account.get('risk_profile', 'moderate')
    signal_risk = signal.get('risk_level', 'medium')

    risk_matrix = {
        'conservative': ['low'],
        'moderate': ['low', 'medium'],
        'aggressive': ['low', 'medium', 'high']
    }

    return signal_risk in risk_matrix.get(risk_profile, ['medium'])

async def _customize_signal_for_account(self, signal: Dict[str, Any], account: Dict[str, Any]) -> Optional[Dict[str, Any]]:
    """Customize signal parameters for specific account"""
    try:
        account_signal = signal.copy()
        account_signal['account_id'] = account['account_id']

        # Adjust lot size based on account balance
        base_lot_size = signal.get('lot_size', 0.01)
        balance_multiplier = account['balance'] / 10000 # Base balance
        account_signal['lot_size'] = round(base_lot_size * balance_multiplier)

        # Adjust risk based on risk profile
        risk_profile = account.get('risk_profile', 'moderate')
        risk_multipliers = {
            'conservative': 0.5,
            'moderate': 1.0,
            'aggressive': 1.5
        }

        multiplier = risk_multipliers.get(risk_profile, 1.0)
        account_signal['lot_size'] *= multiplier

        # Ensure minimum lot size

```

```

        account_signal['lot_size'] = max(0.01, account_signal['lot_size'])

    return account_signal

except Exception as e:
    self.logger.error(f"Error customizing signal for account {account['ac
    return None

# =====
# 9. BACKTESTING ENGINE
# =====

class BacktestingEngine:
    """Comprehensive backtesting with statistical analysis"""

    def __init__(self, db_manager: OptimizedDatabaseManager):
        self.db_manager = db_manager
        self.logger = structlog.get_logger(__name__)

    async def run_backtest(self, strategy_config: Dict[str, Any],
                           start_date: datetime, end_date: datetime) -> Dict[str,
        """Run comprehensive backtest"""
        backtest_uuid = str(uuid.uuid4())

        try:
            # Get historical events
            historical_events = await self._get_historical_events(start_date, end

            # Simulate trading
            trades = []
            portfolio_value = [strategy_config.get('initial_balance', 10000)]

            for event in historical_events:
                # Generate signal based on event
                signal = await self._generate_historical_signal(event, strategy_c

                if signal:
                    # Simulate trade execution
                    trade_result = await self._simulate_trade(signal, event)
                    trades.append(trade_result)

                    # Update portfolio value
                    new_value = portfolio_value[-1] + trade_result.get('pnl', 0)
                    portfolio_value.append(new_value)

            # Calculate statistics
            stats = await self._calculate_backtest_statistics(trades, portfolio_v

```

```

        # Save results
        await self._save_backtest_results(backtest_uuid, strategy_config, sta

        # Generate report
        report = await self._generate_backtest_report(backtest_uuid, stats, t

    return {
        'backtest_uuid': backtest_uuid,
        'statistics': stats,
        'trades': trades,
        'report_path': report
    }

except Exception as e:
    self.logger.error(f"Error running backtest: {e}")
    return {'error': str(e)}

async def _calculate_backtest_statistics(self, trades: List[Dict],
                                         portfolio_values: List[float],
                                         config: Dict[str, Any]) -> Dict[str, A

    """Calculate comprehensive backtest statistics"""
    if not trades:
        return {}

    # Basic statistics
    total_trades = len(trades)
    winning_trades = len([t for t in trades if t.get('pnl', 0) > 0])
    losing_trades = total_trades - winning_trades

    pnl_values = [t.get('pnl', 0) for t in trades]
    total_pnl = sum(pnl_values)

    # Advanced statistics
    win_rate = (winning_trades / total_trades) * 100 if total_trades > 0 else

    winning_pnls = [pnl for pnl in pnl_values if pnl > 0]
    losing_pnls = [pnl for pnl in pnl_values if pnl < 0]

    avg_win = np.mean(winning_pnls) if winning_pnls else 0
    avg_loss = np.mean(losing_pnls) if losing_pnls else 0

    profit_factor = abs(sum(winning_pnls) / sum(losing_pnls)) if losing_pnls

    # Drawdown calculation
    peak = portfolio_values[0]
    max_drawdown = 0

```

```

for value in portfolio_values:
    if value > peak:
        peak = value
    drawdown = (peak - value) / peak * 100
    max_drawdown = max(max_drawdown, drawdown)

# Sharpe ratio (simplified)
if len(pnl_values) > 1:
    sharpe_ratio = np.mean(pnl_values) / np.std(pnl_values) * np.sqrt(252)
else:
    sharpe_ratio = 0

return {
    'total_trades': total_trades,
    'winning_trades': winning_trades,
    'losing_trades': losing_trades,
    'win_rate': win_rate,
    'total_pnl': total_pnl,
    'avg_win': avg_win,
    'avg_loss': avg_loss,
    'profit_factor': profit_factor,
    'max_drawdown': max_drawdown,
    'sharpe_ratio': sharpe_ratio,
    'final_balance': portfolio_values[-1] if portfolio_values else 0
}

# =====
# 10. COMPLIANCE & REGULATORY REPORTING
# =====

class ComplianceManager:
    """Regulatory compliance and audit trail management"""

    def __init__(self, db_manager: OptimizedDatabaseManager):
        self.db_manager = db_manager
        self.logger = structlog.get_logger(__name__)

    async def generate_compliance_report(self, start_date: datetime,
                                         end_date: datetime,
                                         report_type: str = 'monthly') -> Dict[str,
                                         """Generate comprehensive compliance report"""
    try:
        report_uuid = str(uuid.uuid4())

        # Collect compliance data
        trade_data = await self._get_trade_compliance_data(start_date, end_da
        risk_data = await self._get_risk_compliance_data(start_date, end_date

```

```

system_data = await self._get_system_compliance_data(start_date, end_

# Generate report sections
report = {
    'report_uuid': report_uuid,
    'report_type': report_type,
    'period_start': start_date.isoformat(),
    'period_end': end_date.isoformat(),
    'generated_at': datetime.now().isoformat(),
    'trade_summary': await self._generate_trade_summary(trade_data),
    'risk_metrics': await self._generate_risk_metrics(risk_data),
    'system_health': await self._generate_system_health(system_data),
    'violations': await self._check_compliance_violations(trade_data,
    'recommendations': await self._generate_recommendations(trade_dat
}

# Save report
await self._save_compliance_report(report)

# Generate PDF report
pdf_path = await self._generate_pdf_report(report)
report['pdf_path'] = pdf_path

return report

except Exception as e:
    self.logger.error(f"Error generating compliance report: {e}")
    return {'error': str(e)}

async def _check_compliance_violations(self, trade_data: List[Dict],
                                         risk_data: List[Dict]) -> List[Dict[str,
    """Check for compliance violations"""
    violations = []

    # Check position size violations
    for trade in trade_data:
        if trade.get('lot_size', 0) > 1.0: # Example limit
            violations.append({
                'type': 'position_size_violation',
                'trade_id': trade.get('id'),
                'description': f"Position size {trade['lot_size']} exceeds li
                'severity': 'medium'
            })

    # Check risk violations
    for risk_metric in risk_data:
        if risk_metric.get('daily_risk', 0) > 0.05: # 5% daily risk limit

```

```

        violations.append({
            'type': 'daily_risk_violation',
            'date': risk_metric.get('date'),
            'description': f"Daily risk {risk_metric['daily_risk']:.2%} e
            'severity': 'high'
        })

    return violations

# =====
# MAIN SYSTEM ORCHESTRATOR
# =====

class EnhancedCalendarSystem:
    """Main enhanced calendar system orchestrator"""

    def __init__(self, config_path: str = "enhanced_config.yaml"):
        self.config_manager = ConfigManager(config_path)
        self.db_manager = None
        self.event_processor = None
        self.error_recovery = None
        self.notification_system = None
        self.risk_manager = None
        self.portfolio_manager = None
        self.backtesting_engine = None
        self.compliance_manager = None
        self.memory_processor = None

        self.logger = structlog.get_logger(__name__)
        self._running = False

    async def initialize(self):
        """Initialize all system components"""
        try:
            # Load configuration
            config = await self.config_manager.load_config()

            # Initialize database with optimizations
            db_path = config.get('database_path', 'enhanced_calendar.db')
            self.db_manager = OptimizedDatabaseManager(db_path)
            await self.db_manager.initialize()

            # Initialize components
            self.event_processor = AsyncEventProcessor(self.db_manager)
            self.error_recovery = EnhancedErrorRecovery(self.db_manager)
            self.notification_system = NotificationSystem(self.db_manager, config)
            self.risk_manager = RiskManager(self.db_manager)

```

```

        self.portfolio_manager = PortfolioManager(self.db_manager)
        self.backtesting_engine = BacktestingEngine(self.db_manager)
        self.compliance_manager = ComplianceManager(self.db_manager)
        self.memory_processor = MemoryOptimizedProcessor()

        # Start hot configuration reloading
        self.config_manager.start_hot_reload()

        self.logger.info("Enhanced calendar system initialized successfully")

    except Exception as e:
        self.logger.error(f"Error initializing system: {e}")
        raise

    async def start(self):
        """Start the enhanced calendar system"""
        try:
            await self.initialize()

            # Start all components
            await self.event_processor.start()
            await self.notification_system.start()

            self._running = True
            self.logger.info("Enhanced calendar system started")

            # Start main processing loop
            await self._main_loop()

        except Exception as e:
            self.logger.error(f"Error starting system: {e}")
            await self.error_recovery.handle_error(e, {'component': 'main_system'})

    async def stop(self):
        """Stop the enhanced calendar system"""
        self._running = False

        if self.event_processor:
            await self.event_processor.stop()
        if self.notification_system:
            await self.notification_system.stop()

        self.logger.info("Enhanced calendar system stopped")

    async def _main_loop(self):
        """Main system processing loop"""
        while self._running:

```

```

try:
    # Monitor system health
    await self._monitor_system_health()

    # Process any pending calendar files
    await self._process_pending_files()

    # Monitor for triggered events
    await self._monitor_event_triggers()

    # Sleep before next iteration
    await asyncio.sleep(15) # 15-second cycle

except Exception as e:
    self.logger.error(f"Error in main loop: {e}")
    await self.error_recovery.handle_error(e, {'component': 'main_loo
    await asyncio.sleep(60) # Wait longer after error

async def _monitor_system_health(self):
    """Monitor system health and performance"""
    try:
        # Check memory usage
        memory_info = psutil.virtual_memory()
        if memory_info.percent > 80:
            await self.notification_system.send_critical_alert(
                "High Memory Usage",
                f"System memory usage is {memory_info.percent}%"
            )

        # Check database performance
        start_time = asyncio.get_event_loop().time()
        async with self.db_manager.get_connection() as conn:
            await conn.execute("SELECT 1")
        db_response_time = asyncio.get_event_loop().time() - start_time

        if db_response_time > 1.0: # Slow database response
            await self.notification_system.send_critical_alert(
                "Slow Database Response",
                f"Database response time: {db_response_time:.2f}s"
            )

    except Exception as e:
        self.logger.error(f"Error monitoring system health: {e}")

# Usage Example
async def main():

```

```

"""Example usage of the enhanced calendar system"""
system = EnhancedCalendarSystem("enhanced_config.yaml")

try:
    await system.start()
except KeyboardInterrupt:
    await system.stop()

if __name__ == "__main__":
    # Configure structured logging
    structlog.configure(
        processors=[
            structlog.stdlib.filter_by_level,
            structlog.stdlib.add_logger_name,
            structlog.stdlib.add_log_level,
            structlog.stdlib.PositionalArgumentsFormatter(),
            structlog.processors.TimeStamper(fmt="iso"),
            structlog.processors.StackInfoRenderer(),
            structlog.processors.format_exc_info,
            structlog.processors.JSONRenderer()
        ],
        context_class=dict,
        logger_factory=structlog.stdlib.LoggerFactory(),
        wrapper_class=structlog.stdlib.BoundLogger,
        cache_logger_on_first_use=True,
    )

    asyncio.run(main())

```